zgz-pileup — how the code works

This document follows the eight scripts in order and says, for each one, what it computes, with which rule, where that rule is in the code, and how you can check it yourself. Every number that is fixed by hand is listed with its file. Everything else is measured from the data and printed by the scripts. The report explains why the method is built this way; this document is about what the code does.

Code is referred to as `file.py: function` (or `CONSTANT`). Times are in samples of 128 ns unless said otherwise; amplitudes in ADC counts.

First, run `python scripts/checks.py` (section 10). It checks the three things most worth doubting, the pairing of records with `out.txt`, the fitter and the energies of resolved pulses, on your own data and without using any of our numbers.

---

1. Reading the data — `01_read_data.py`

`c<ch>.txt` → records (`io.py: read_channel`). The file is read in chunks of 5 million lines with pandas. A blank line ends a record. For every record the script keeps:

- the heights, divided by the ADC step and rounded to integers (`int16`). The step (e.g. 254.42 for channel 1) is found from the spacing of the distinct height values (`io.py: _adc_step`), so the stored values are the 8-bit ADC counts;
- its length (number of samples);
- the maximum of the third column (VST filter), used only to pair records with `out.txt`.

The sampling period is the median spacing of the first column (0.128 µs). Records are stored back to back in one array with their offsets and lengths (`io.py: Records`); `Records.wave(i)` returns record `i`.

Check: `records.npz` holds `adc`, `offset`, `length`, `fmax`. The number of records equals the number of blank-line-separated blocks of `c<ch>.txt`.

Records ↔ `out.txt` rows (`io.py: align_times`). Each row of `out.txt` belongs to one record, in the same order, but the k-th row of a channel is not always the k-th record. In RUN1:

- `c<ch>.txt` has 88–96 more records than `out.txt` has rows: the last records of each file have no row;
- in one or two places per channel the row numbering slips (a row with no record), and every row after that point belongs to the next record (channel 1: after record 334,823).

Pairing record k with row k therefore gives the wrong time to every record after the first slip (84 % of the channel-1 records). The code pairs record k with row k + offset. The offset starts at 0 and changes only where the energies show a slip:

1. a pair is *certain* when the record's VST maximum equals the row's energy (within 0.05);
2. a slip shows up as 10 consecutive pairs that disagree (`run=10`). Single disagreements are normal: 1–11 % of the pairs differ in energy, depending on the channel;
3. at a slip, the new offset is the shift within ±200 rows (`max_shift`) that makes most of the next 60 records agree (`window=60`);

4. records between two certain pairs with the same offset are paired by position, whatever their energy.

The time of a record is then the time column of its row (`times.npz: time`; NaN for records without a row).

Check: `checks.py`, check 1, prints the record and row counts, the energy agreement of naive and slip-corrected pairing, and where the slips are.

---

2. Classes and clean pulses — `02_clean_pulses.py`

Measurements of every record (`pulses.py: record_features`). With `x` the record minus its baseline:

Quantity	Definition
baseline	mean of the first 12 samples (`N_BASELINE`)
baseline peak-to-peak	max − min of those 12 samples
`mx`	highest sample (saturation test)
area	sum of `x` over the whole record
amplitude (smoothed)	maximum of `x` after a 3-sample moving average (`W`)
rise	samples between the 10 % and 90 % crossings of the smoothed pulse
`t50`	first sample where the smoothed pulse reaches 50 % of its maximum
triggers	number of rises seen by the selection filter (below)

The **selection filter** is the TM fast filter (section 4) with a short window, `L = 4` (`SEL_L`), so that it re-arms soon after a rise. Its threshold is 5.5 (`SEL_SIGMA`) times its noise, which is the clipped standard deviation of the filter on the pre-trigger samples of records with a flat baseline (`pulses.py: selection_noise`). Triggers within 20 samples (`MERGE`) of the first one belong to the same rise; triggers in the last 4 samples are ignored, because there the filter's look-ahead window runs past the end of the record.

Classes (`pulses.py: classify`), first rule that applies:

Class	Rule
saturated	`mx` ≥ the ADC ceiling (the largest value in the file, 127)
baseline	baseline mean beyond ±1.5 ADC (`BASE_MAX_ABS`) or peak-to-peak > 3 ADC (`BASE_MAX_PTP`): the record starts on the tail of an earlier pulse
no pulse	the selection filter never fires
2nd trigger	the selection filter fires again after the pulse's own rise: pile-up
slow rise	10–90 % rise longer than 20 samples (`MAX_RISE`): pile-up within the rise
off diagonal	area more than 3 robust standard deviations (`BAND_K`) above the single-pulse curve, or below 0.3 of it (`AREA_FLOOR`)
clean	none of the above

The single-pulse curve (`pulses.py: band_fit`). Single pulses lie on a smooth curve of area against amplitude. The curve is followed outward from the most populated amplitude bin; in each bin, only records whose area/amplitude is within 10 % of the value predicted from the

neighbouring bins are used (median and robust spread), so the curve stays on the main population. A quadratic is fitted to the bin medians. The cut is one-sided, since pile-up adds area; records below the curve are kept (the second pulse shape; report, Section 3.4).

Pulse shape (`pulses.py: aligned_stack, shape_from_stack`). Up to 200,000 clean pulses (`N_SHAPE`, pooled over the runs of the group) are divided by their amplitude and shifted so that their `t50` falls on the same sample. The shape is the median at each sample, normalised to a maximum of 1 (left undefined where fewer than 50 records reach). This is done twice: first dividing by the smoothed maximum, then by the template amplitude of the first pass.

Template amplitude (`pulses.py: template_amplitudes`). The least-squares scale of the shape over the recorded samples of the pulse, $A = \Sigma\, x_j\, T_j \,/\, \Sigma\, T_j^2$, with T aligned on the pulse's `t50`. This is the amplitude of a clean pulse in `events.csv`.

Hold-out. The last 10 % (`HOLDOUT`) of the clean pulses of each run, in record order, are never used for the templates. Step 7 tests the fitter on them.

Several runs. Classes are decided run by run. The shape is learned from the pooled clean pulses of all runs of a channel (`POOL_RUNS`), and the photopeak of each run is printed: a difference above 1 % means different gains, and runs should not be pooled.

Check: figures `02_clean_selection.png` and `03_record_classes.png` (run folder) show the area band and random records of every class; `04_clean_pulses.png` (model folder) shows 100,000 clean pulses divided by their amplitude, which should form one narrow band.

---

3. Tail and undershoot — `03_tail.py`

Records end when the pulse has fallen to about 4 ADC, long before it returns to baseline, so no record contains the end of the tail or the undershoot. They are measured from **pairs of consecutive records** (`tail.py: baseline_pairs`): when record *i* starts while the pulse of record *i−1* is still decaying, the baseline of *i* is that pulse's value at that moment.

Pairs are kept when the earlier record is a clean pulse of 15–40 ADC (`AMIN`, `AMAX`), with no other record in the 1000 µs before it (`ISOLATION_US`), and the two records do not overlap. For each pair:

- time after the earlier pulse's reference point, in samples: $k = (t_i - t_{i-1}) / \Delta t + (12 - 1)/2 - (t50_{i-1} + D)$ (the baseline is the mean of the first 12 samples, centred 5.5 samples into the record);
- value per unit amplitude: $y = (\text{baseline}_i - \text{zero}) / A_{i-1}$, where *zero* is the median baseline of records that follow a gap of more than 5 ms (`ZERO_GAP_US`), i.e. the true baseline.

The pairs are binned in 1 µs bins (`BIN = 8` samples); bins with at least 100 pairs (`MIN_PAIRS`) keep their median, and the medians are smoothed with a Savitzky–Golay filter of 9 bins (`tail.py: tail_curve`). If the curve ends before the longest record, it is continued with the exponential decay of its last 50 µs.

Check: `05_tail_undershoot.png`, panel (a): every pair is a dot. The curve must join the clean-pulse shape where both exist; the zero crossing and the minimum are printed.

---

4. TM trigger — `04_tm_params.py`

Fast filter (`fitter.py: compute_fastfilter`): f_i = (mean of the L samples from i on) − (mean of the L samples before i). **Trigger** (`fitter.py: find_triggers`): sample i where f_i ≥ threshold and the two previous values are below it. Triggers in the last L samples of a record are ignored (`tm.py: triggers`), for the reason given in section 2.

Choice of L and threshold (`tm.py: characterise`), on 20,000 training pulses (clean, not held out):

- the noise of the filter is measured on the pulses minus their fitted shape, after sample 120 (only noise is left there);
- L is chosen among 1–32 samples (`L_CANDIDATES`) as the one with the best ratio of the filter's maximum on the unit pulse to its noise, among those that fit before the pulse: L may not exceed the 1st percentile of the pulse onset minus 3 samples (the pre-trigger is short);
- the threshold is the lowest of 2–12 times the filter noise at which at most 0.2 % (`MULTI_MAX`) of the clean pulses give more than one trigger;
- `peak2trig` is the median distance from the first trigger to the reference point; it places the first guess of each pulse.

Check: `06_tm_settings.png`: panel (a) the signal-to-noise against L, with the lengths that do not fit greyed out; panel (b) the fraction of clean pulses with 0, 1 or more triggers against the threshold.

---

5. Templates — `05_train_tm.py`

Reference point. Each pulse is described relative to its reference point, D samples after `t50` (the centre of the flat top; D from `pulses.py: shape_times`). The maximum sample is not used, because the flat top is several samples long and quantised in steps of one ADC count.

Point models (`fitter.py: PointModels`). At every offset k from the reference point, the template value per unit amplitude is a quadratic in the amplitude,

$h_k(A) = a0_k + a1_k A + a2_k A^2$,

so a pulse of amplitude A at position p contributes $A \cdot h_{j-p}(A)$ to sample j; between integer offsets the value is interpolated linearly. A record is fitted as a sum of such pulses.

Training (`tm.py: head_coefficients`). 250,000 training pulses (`tm.N_TRAIN`, pooled over runs) are aligned on their reference points, from 16 samples before `t50` (`PRE_SAMPLES`) to 32 samples (`BLEND`) after the start of the measured tail. Where a record has already ended, the pulse is continued with its amplitude times the mean shape (`pulses.py: padded_batch`). At each offset, (a0, a1, a2) is the least-squares fit of x_k / A against A over all training pulses.

Joining the tail (`tm.py: build_coefficients`). From the start of the measured tail (step 3) the template is the tail itself, independent of A, blended linearly with the trained head over 32 samples, and it continues to 50 samples past the longest record. The template therefore includes the undershoot of every pulse.

Residual threshold. 400 held-out clean pulses (`N_NOISE`) are fitted as single pulses; the threshold for adding pulses in the fit (section 6) is the 95th percentile of the largest value of

(model − data) in those fits.

Check: `05_tail_undershoot.png`, panel (b): the full template, from the head to the end of the tail. `summary.json`, key `tm`, `single_fit_amplitude_ratio_tm_over_template`: the TM amplitude of single pulses divided by their template amplitude, which should be 1.00.

---

6. The fit — `zgzpileup/fitter.py`

The fitter is the template-matching code of LaBr3Wave (github.com/pkuNucExp/LaBr3Wave), as modified in our LaBr3(Ce) work. The fitting code is unchanged; only plotting and multiprocessing helpers were removed (header of `fitter.py`). For one record (`tm.py: Model.fit`):

1. **Triggers.** The fast filter gives the trigger positions. No trigger: no fit.
2. **Window.** From the first trigger + `rangeleft` to the last trigger + `rangeright` (the template length), clipped to the record.
3. **First fit.** One pulse per trigger, starting at trigger + `peak2trig` with the record value there as amplitude. Amplitudes and positions of all pulses are fitted together by bounded least squares (`scipy.optimize.least_squares`, analytic Jacobian), amplitude in [0, 1.2 × ceiling], positions such that every pulse overlaps the window. Saturated samples are left out of the residuals. The quality is $\chi^2/\mathrm{ndf} = \Sigma(\mathrm{model} - \mathrm{data})^2 / (\mathrm{samples} - \mathrm{parameters})$.
4. **Residual search** (`WaveformFitter.fit_window`). Peaks of (model − data) higher than the residual threshold are candidate pulses, tried from the highest down. A candidate is added, and all pulses refitted, only if χ^2/ndf falls below η times its previous value, with $\eta = 1 - (1 - \eta 0) \cdot L0 / L$, $\eta 0 = 0.7$ (`tm.ETA0`), L0 the template length and L the window length. At most three pulses are added per record.

The search looks for peaks of model − data, as in LaBr3Wave. A missing pulse makes the fit overshoot on one side of it and undershoot on the other, so the search finds it either way. It matters most at short delays: without it, pairs 2–4 µs apart are found in 16 % of cases instead of 84 %.

Check: `checks.py`, check 2, builds records with known pulses and prints the true and fitted pulses side by side (figure `13_checks_fitter.png`).

---

7. Calibration — `06_calibrate.py`

$E = c \cdot A$, with c = `PEAK_KEV` / (photopeak position of the clean template amplitudes). The photopeak position is the centre of a Gaussian fitted three times to the top of the peak, each time over ±1.2 σ of the previous fit (`pulses.py: photopeak`). As a check, the script finds the sum peak (two photopeak pulses detected together) between 1.7 and 2.3 times the photopeak, which should be at about twice the photopeak energy.

---

8. Benchmark — `07_benchmark.py`

Pairs of held-out clean pulses are added with a delay drawn log-uniformly between 0.2 and 100 µs. Each pulse is continued after the end of its record with its amplitude times the template tail plus Gaussian noise of the baseline rms (`tm.py: continued`), and the sum is rounded to integers. The pairs are fitted exactly like real records. A true pulse is **found** when a fitted pulse lies within 4 samples (`TOL`) and its amplitude within 10 % (`AMP_TOL`). Errors are robust standard deviations (1.4826 × median absolute deviation). 5,000 pairs (`N_BENCHMARK` in `config.py`) and up to 3,000 single held-out pulses are used.

---

9. Recovery and the result table — `08_recover.py`

Records of the classes 2nd trigger, slow rise, off diagonal and baseline are fitted (section 6). Clean pulses keep their template amplitude and are not fitted. Saturated records get a time from their first trigger and no amplitude. For each pulse:

- `time_in_record_us` = (position − D) × Δt, the 50 % rise of the pulse;
- `time_s` = record time from `out.txt` + (position − D − `t50` median) × Δt. The record time belongs to the trigger of the record, which sits at the median `t50` of the clean pulses;
- `energy_kev` = c × amplitude;
- `flag`: `outside_record` if the fitted position is outside the record (typically the earlier pulse whose tail the record starts on), `amplitude_at_bound` if the amplitude hit a fit bound, `saturated`.

Reproducibility. All random choices (samples for the shape, training and hold-out pulses, benchmark pairs) use `SEED` from `config.py`. Running the scripts twice gives identical files.

---

10. Checks you can run — `scripts/checks.py`

```
python scripts/checks.py --channels 1
```

prints, for each run and channel:

1. **Records and `out.txt`**: the counts, the energy agreement of record k with row k (naive) and with the paired row, and where the numbering slips. For RUN1 channel 1:

`records in c1: 2,036,169 rows of channel 1 in out.txt: 2,036,074 difference: +95 record k paired with row k (naive), energy agrees in: records 0 – 334,823: 98.6% records 334,824 – 2,036,073: 0.1% paired by step 1 (row = record + offset): 2,036,073 records (99.995%), energy agrees in 98.7%`

1. **The fitter on known pulses**: eight records built from held-out clean pulses (one pulse, pairs 20, 6, 3 and 1.5 µs apart, three pulses, a small pulse before a large one, a small one on a tail), fitted like real records. For RUN1 channel 1, every pulse is found; the positions agree within 0.2 µs and the amplitudes within 1.3 ADC (4 %), except for the pair about 1.3 µs apart, whose amplitudes come out as 28 and 35 instead of 31 and 31 (the benchmark shows the same limit below 2 µs).

2. **Energies of resolved pile-up**: the photopeak of clean pulses and of pulses resolved from pile-up records. For RUN1 channel 1: 31.22 ADC with 6.0 % FWHM against 31.28 ADC with 5.9 % FWHM.

---

11. Fixed settings

Setting	Value	Where
baseline samples	12	`pulses.py: N_BASELINE`
flat baseline	mean within ±1.5 ADC, peak-to-peak ≤ 3 ADC	`pulses.py: BASE_MAX_ABS`, `BASE_MAX_PTP`
selection filter	L = 4, threshold 5.5 × noise, merge 20 samples	`pulses.py: SEL_L`, `SEL_SIGMA`, `MERGE`
largest rise of a clean pulse	20 samples (2.6 µs)	`pulses.py: MAX_RISE`
area band	+3 robust σ above the curve, floor 0.3 × curve	`pulses.py: BAND_K`, `AREA_FLOOR`
hold-out	last 10 % of the clean pulses of each run	`02_clean_pulses.py: HOLDOUT`
pulses for the shape	200,000	`02_clean_pulses.py: N_SHAPE`
tail pairs	earlier pulse 15–40 ADC, isolated 1000 µs; 1 µs bins, ≥ 100 pairs	`tail.py`
fast-filter lengths tried	1–32 samples	`tm.py: L_CANDIDATES`
trigger threshold rule	≤ 0.2 % of clean pulses with > 1 trigger	`tm.py: MULTI_MAX`
template range	16 samples before `t50` to 50 samples after the longest record	`tm.py: PRE_SAMPLES`, `03_tail.py`
training pulses	250,000	`tm.py: N_TRAIN`
head–tail blend	32 samples	`05_train_tm.py: BLEND`
pulse acceptance	η0 = 0.7	`tm.py: ETA0`
benchmark match	4 samples, 10 % in amplitude	`07_benchmark.py: TOL`, `AMP_TOL`